

# Grundlagen der Informatik & Automatentheorie

---

EVA-Prinzip - Zahlensysteme - Boolesche Algebra - Von-Neumann - Automaten -  
Turingmaschine

*Lernskript zur Vorbereitung auf die mündliche Prüfung*

# Inhaltsverzeichnis

|                                                              |                                           |
|--------------------------------------------------------------|-------------------------------------------|
| 1. Grundlagen & EVA-Prinzip .....                            | 4                                         |
| Bestandteile von Informationen .....                         | 4                                         |
| 2. Prozesse .....                                            | 4                                         |
| Prozessabbild im Speicher .....                              | 4                                         |
| Prozesserzeugung (Unix-Modell) .....                         | 5                                         |
| Eigenschaften von Prozessen .....                            | 5                                         |
| 3. Informationsdarstellung & Zahlensysteme .....             | 6                                         |
| Datenhierarchie .....                                        | 6                                         |
| Stellenwertsysteme .....                                     | 6                                         |
| Umwandlung: Dezimal -> Fremdsystem (Division mit Rest) ..... | 7                                         |
| Umwandlung: Fremdsystem -> Dezimal .....                     | 7                                         |
| 4. Codierung von Zeichen .....                               | 7                                         |
| 5. Zweierkomplement (negative Zahlen) .....                  | 7                                         |
| Subtraktion in fremder Basis (Beispiel Basis 7) .....        | 8                                         |
| 6. Gleitpunktverfahren (Gleitkommazahlen) .....              | 8                                         |
| Beispiel: 11,5 in 16-Bit-Gleitkomma .....                    | 8                                         |
| 7. Boolesche Algebra .....                                   | 9                                         |
| Wahrheitstabellen (AND, OR, XOR) .....                       | <b>Fehler! Textmarke nicht definiert.</b> |
| Wichtige Rechengesetze .....                                 | 9                                         |
| Disjunktive Normalform (DNF) .....                           | 10                                        |
| Vereinfachung (algebraisch) .....                            | 10                                        |
| Karnaugh-Veitch-Diagramm (KV-Diagramm) .....                 | 10                                        |
| Gatterschaltungen & Schaltbilder .....                       | 10                                        |
| 8. Rechnerarchitektur (Von-Neumann) .....                    | 11                                        |
| Register & Sonderregister .....                              | 11                                        |
| Steuerwerk - der Befehlszyklus .....                         | 12                                        |
| 9. CISC, RISC & Pipelining .....                             | 12                                        |
| Phasen-Pipelining .....                                      | 12                                        |
| 10. Speicherarten .....                                      | 13                                        |
| Pufferspeicher (Caches) .....                                | 13                                        |
| Virtueller Speicher .....                                    | 13                                        |
| Wechselstrategien (Verdraengung) .....                       | 13                                        |
| 11. Bus & Geraeteverwaltung .....                            | 14                                        |
| Bus (Sammelleitung) .....                                    | 14                                        |
| Geraeteverwaltung (E/A-Systeme) .....                        | 14                                        |
| 12. Softwarekomponenten & Betriebssysteme .....              | 15                                        |
| Vom Quellcode zum Programm .....                             | 15                                        |
| Prozessorverwaltung (Scheduling) .....                       | 15                                        |
| 13. Dateisysteme .....                                       | 16                                        |
| Deskriptoren .....                                           | 16                                        |
| Hierarchische Dateisysteme .....                             | 16                                        |

|                                                       |    |
|-------------------------------------------------------|----|
| Dateiorganisation .....                               | 16 |
| Dateisysteme unter Unix .....                         | 17 |
| Arten von Dateien.....                                | 17 |
| 14. Programmiersprachen .....                         | 17 |
| Problemorientierte (hoehere) Sprachen .....           | 17 |
| Maschinenorientierte Sprachen.....                    | 18 |
| Adressierungsarten .....                              | 18 |
| 15. Sprachen, Grammatiken & Beschreibungsformen.....  | 18 |
| Grammatik $G = (T, V, S, P)$ .....                    | 18 |
| Backus-Naur-Form (BNF) .....                          | 19 |
| Regulaere Ausdruecke .....                            | 19 |
| Chomsky-Hierarchie .....                              | 19 |
| 16. Automaten.....                                    | 19 |
| Deterministischer endlicher Automat (DEA).....        | 20 |
| Determinismus - die vier Begriffe .....               | 20 |
| Nichtdeterministischer Automat (NEA).....             | 21 |
| Umwandlung NEA -> DEA .....                           | 21 |
| Grammatik aus einem endlichen Automaten ableiten..... | 21 |
| 17. Turingmaschine .....                              | 21 |
| Konfiguration .....                                   | 21 |
| 18. Registermaschine .....                            | 22 |
| Operationen.....                                      | 22 |
| Programmaufbau.....                                   | 22 |
| Beispiel: Addition (Tupelschreibweise).....           | 22 |

# 1. Grundlagen & EVA-Prinzip

Jede Datenverarbeitung folgt demselben Grundschemata, dem EVA-Prinzip: Eingabe -> Verarbeitung -> Ausgabe.

- **Eingabe:** Daten und Programme gelangen ins System  
(früher Lochkarten, heute Tastatur/Maus oder Übertragung von anderen Anlagen).
- **Verarbeitung:** Findet in der Zentraleinheit (CPU) statt.  
Aktive Eingabedaten (Programme) steuern die Verarbeitung der passiven Eingabedaten.
- **Ausgabe:** Die errechneten Daten werden an die Außenwelt gegeben (Bildschirm, Drucker). Die Ausgabe kann wiederum Eingabe für den nächsten Schritt sein.

## Bestandteile von Informationen

| Ebene                                                                                                                                                                                              | Bedeutung                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Syntaktisch                                                                                                                                                                                        | Struktur der Bausteine - wie sind die Daten aufgebaut?           |
| Semantisch                                                                                                                                                                                         | Bedeutung der Information - wofür stehen die Zeichen?            |
| Pragmatisch                                                                                                                                                                                        | Zweck / Reaktion - welche Wirkung soll die Information erzielen? |
| <b>Prüfungstipp</b><br>Diese drei Ebenen tauchen später bei den Programmiersprachen wieder auf (Syntax = Satzbau, Semantik = Bedeutung, Pragmatik = erwartete Aktion). Merke dir den Zusammenhang. |                                                                  |

## 2. Prozesse

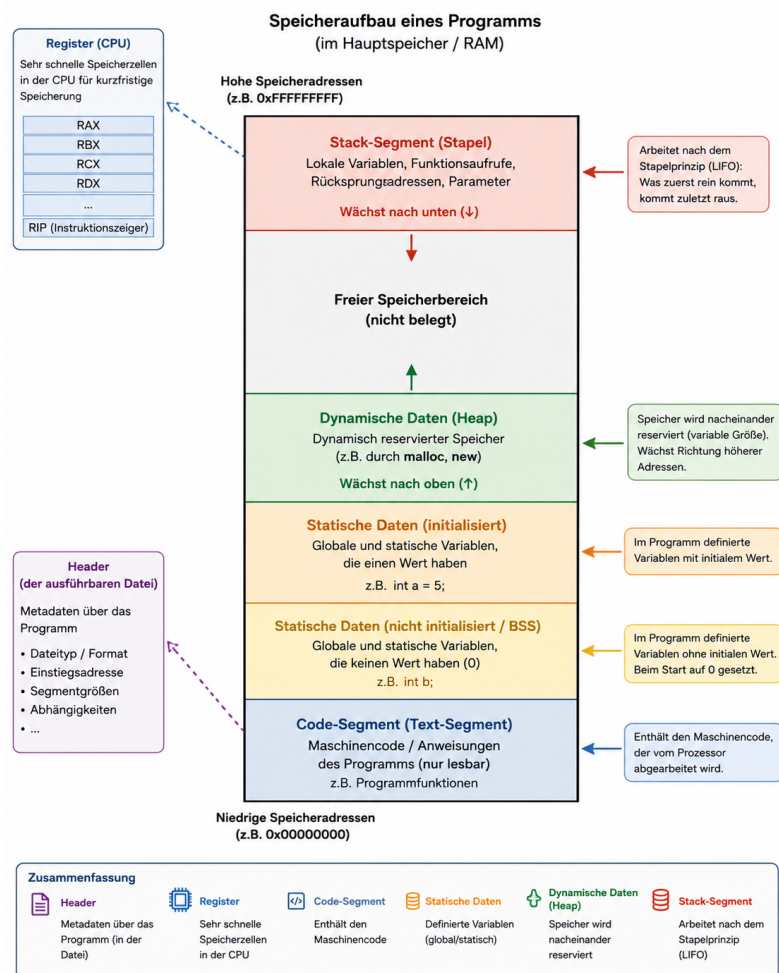
Ein Prozess ist der Vorgang einer algorithmisch ablaufenden Informationsverarbeitung.

Innerhalb des Betriebssystems ist ein Prozess ein Vorgang, der durch ein Programm gesteuert wird und einen Prozessor benötigt.

| Programm                                         | Prozess                                        |
|--------------------------------------------------|------------------------------------------------|
| Verarbeitungsvorschrift (Algorithmus) - statisch | Verarbeitungsvorgang - die laufende Ausführung |

## Prozessabbild im Speicher

- Header: Metadaten über das ausführbare Programm.
- Register: Speicherzellen auf der CPU zur kurzfristigen Speicherung.
- Code-Segment: Enthält den Maschinencode.
- Statische Daten: Definierte Variablen.
- Dynamische Daten (Heap): Speicher wird nacheinander reserviert (variable Größe).
- Stack-Segment: Arbeitet nach dem Stapelprinzip (LIFO - was zuerst rein kommt, kommt zuletzt raus).



## Prozesserzeugung (Unix-Modell)

- `fork()`: Klont den Vaterprozess (erzeugt einen Kindprozess).
- `exec()`: Lädt ein Programm - die Verarbeitungsvorschrift wird ins Code-Segment eingetragen.
- `wait()`: Der Vaterprozess wartet auf das Kind.
- `exit()`: Beendet den Prozess und gibt einen Return-Code zurück (0 = alles OK), Speicher wird freigegeben.
- **Zombieprozess**: Belegt nach dem Absturz/Ende weiterhin Speicher, weil der Return-Code noch nicht abgeholt wurde.

## Eigenschaften von Prozessen

- Laufen normalerweise geordnet ab (gestartet, überwacht, beendet).
- Sind unterbrechbar und später fortsetzbar.
- Sind durch andere Prozesse beeinflussbar.

### 3. Informationsdarstellung & Zahlensysteme

---

#### Datenhierarchie

| Ebene    | Hierarchie (klein -> gross)                           |
|----------|-------------------------------------------------------|
| Logisch  | Zeichen -> Feld/Begriff -> Satz -> Datei -> Datenbank |
| Physisch | Bit -> Byte -> Block                                  |

- Bit: kleinste Einheit - an/aus, 0/1.
- Byte: 8 Bit = 256 Kombinationen ( $2^8$ ). Jedes Bit weniger halbiert die Anzahl der Kombinationen.
- **Block: speichert** Begriffe oder Sätze; Datei: mehrere Blöcke; Datenbank: Sammlung von Dateien.

**Aktive Daten:** steuern die Arbeit des Computers (Anweisungen / Befehle).

**Passive Daten:** werden dem Verarbeitungsprozess unterworfen (z. B. Stammdaten einer Kundendatei).

#### Stellenwertsysteme

In einem Stellenwertsystem hängt der Wert einer Ziffer von ihrer Stelle (Potenz der Basis) ab.

| System       | Basis | Ziffernvorrat |
|--------------|-------|---------------|
| Dual (binär) | 2     | 0, 1          |
| Oktal        | 8     | 0-7           |
| Dezimal      | 10    | 0-9           |
| Hexadezimal  | 16    | 0-9, A-F      |

Beispiel Dezimal:  $164 = 1 \cdot 10^2 + 6 \cdot 10^1 + 4 \cdot 10^0 = 100 + 60 + 4$ . Dieselbe Ziffernfolge 164 als Oktalzahl:  $1 \cdot 8^2 + 6 \cdot 8^1 + 4 \cdot 8^0 = 64 + 48 + 4 = 116$ .

## Umwandlung: Dezimal -> Fremdsystem (Division mit Rest)

Man teilt fortlaufend durch die Zielbasis und liest die Reste von unten nach oben. Voraussetzung: Der Ausgangswert muss zur Basis 10 vorliegen.

| Rechnung | Ergebnis | Rest |
|----------|----------|------|
| 164 : 8  | 20       | 4    |
| 20 : 8   | 2        | 4    |
| 2 : 8    | 0        | 2    |

Reste von unten nach oben gelesen: 244 zur Basis 8. Beispiel Dual: 164 in Dual ergibt 10100100.  
Probe:  $128 + 32 + 4 = 164$ .

## Umwandlung: Fremdsystem -> Dezimal

Jede Ziffer wird mit ihrer Stellenwertigkeit (Basis hoch Position) multipliziert und alles aufsummiert.

Beispiel:  $10100100 \text{ (Dual)} = 1 * 2^7 + 1 * 2^5 + 1 * 2^2 = 128 + 32 + 4 = 164$ .

### Schnellumrechnung über Gruppen

Binär <-> Oktal: Bits in 3er-Gruppen. Binär <-> Hexadezimal: Bits in 4er-Gruppen. Beispiel: 1010 0100 = A4 (hexadezimal).

### Fehlerprüfung

Eine Ziffer darf nie  $\geq$  Basis sein. 378 zur Basis 8 ist ungültig, weil im Oktalsystem nur Ziffern 0-7 erlaubt sind.

## 4. Codierung von Zeichen

| Code    | Beschreibung                                                                                                                          |
|---------|---------------------------------------------------------------------------------------------------------------------------------------|
| ASCII   | American Standard Code for Information Interchange. Ursprünglich 7 Bit + 1 Paritätsbit; das 8. Bit erlaubt heute zusätzliche Zeichen. |
| EBCDIC  | Extended Binary Coded Decimal Interchange Code. Nutzt ein volles Byte je Zeichen (Zonen- und Ziffernteil).                            |
| Unicode | Internationaler Standard - ein eindeutiger Code pro Schriftzeichen, weltweit.                                                         |

## 5. Zweierkomplement (negative Zahlen)

Das Zweierkomplement stellt negative Zahlen dar, damit ein Rechner nur addieren muss (Subtraktion = Addition des Negativen).

**Bildung in zwei Schritten:** 1. Komplementbildung: jedes Bit stellenweise ins Gegenteil umkehren (Einerkomplement). 2. Addition von 1.

Beispiel: Aus 10110010 wird durch Invertieren 01001101, plus 1 ergibt 01001110.

Entsteht ein Übertrag über die höchste Stelle hinaus (z. B. Ergebnis 100000000 bei 8 Bit), spricht man von einem Wertebereichsüberlauf.

#### **Merksatz**

Ein Rechner kann eigentlich nur addieren. Subtraktion wird realisiert, indem man das Zweierkomplement des Subtrahenden bildet und addiert.

### **Subtraktion in fremder Basis (Beispiel Basis 7)**

Aufgabe  $125 - 34$  in Basis 7: Beide Zahlen erst in Basis 7 umwandeln ( $125 = 236$ ,  $34 = 46$  in Basis 7), dann das Komplement des Subtrahenden bilden, Stellen angleichen und addieren. Ergebnis: 91.

## **6. Gleitpunktverfahren (Gleitkommazahlen)**

Zur Darstellung gebrochener Zahlen wird die Zahl in Vorzeichen, Mantisse und Exponent zerlegt und normalisiert (führende 0, der erste Nachkommawert ist eine 1).

Die Nachkommastellen im Dualsystem haben die Wertigkeiten  $2^{-1} = 0,5$ ;  $2^{-2} = 0,25$ ;  $2^{-3} = 0,125$  usw.

### **Beispiel: 11,5 in 16-Bit-Gleitkomma**

Gegeben: 16 Bit für Vorzeichen + Mantisse, davon 4 Bit für den Exponenten, Bias = 8.

- 1. Dezimal -> Dual:  $11,5 = 1011,1$  (dual)
- 2. Normalisieren:  $1011,1 = 0,10111 \cdot 2^4$  (Komma um 4 Stellen verschoben)
- 3. Exponent mit Bias:  $4 + 8 = 12 = 1100$  (dual)
- 4. Zusammensetzen: Vorzeichen 0 | Mantisse 10111000... | Exponent 1100

#### **Vorsicht beim Umrechnen der Nachkommastellen**

Nachkommastellen werden durch fortlaufende Multiplikation mit 2 ermittelt (der ganzzahlige Anteil ist die nächste Bit-Stelle). Bei wiederkehrendem Muster muss man abbrechen - sonst läuft es endlos.



## 7. Boolesche Algebra

### Logikgatter (Schaltsymbole & Wahrheitstabelle)

#### AND

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

#### OR

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

#### NOT

| A | Y |
|---|---|
| 0 | 1 |
| 1 | 0 |

#### NAND

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

#### NOR

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

#### XOR

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

#### XNOR

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Abb.: Logikgatter – Wahrheitstabellen

Die boolesche Algebra ist die Grundlage der Digitaltechnik. Sie rechnet mit den Wahrheitswerten 0 (falsch) und 1 (wahr).

| Operation   | Symbol            | Bedeutung | Ergebnis = 1, wenn ...            |
|-------------|-------------------|-----------|-----------------------------------|
| Negation    | NOT A / $\bar{A}$ | NOT       | A = 0                             |
| Konjunktion | A AND B           | AND       | beide Eingänge 1 sind             |
| Disjunktion | A OR B            | OR        | mindestens ein Eingang 1 ist      |
| Antivalenz  | A XOR B           | XOR       | die Eingänge unterschiedlich sind |

### Wichtige Rechengesetze

| Gesetz            | Regel                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Assoziativ        | $A \text{ AND } (B \text{ AND } C) = (A \text{ AND } B) \text{ AND } C$ (analog für OR)                                                               |
| Kommutativ        | $A \text{ AND } B = B \text{ AND } A$ (analog für OR)                                                                                                 |
| Distributiv       | $A \text{ AND } (B \text{ OR } C) = (A \text{ AND } B) \text{ OR } (A \text{ AND } C)$                                                                |
| Absorption        | $A \text{ AND } (A \text{ OR } B) = A$                                                                                                                |
| Neutrale Elemente | $A \text{ AND } 1 = A$ ; $A \text{ OR } 0 = A$                                                                                                        |
| Komplement        | $A \text{ AND } \text{NOT } A = 0$ ; $A \text{ OR } \text{NOT } A = 1$                                                                                |
| Doppelnegation    | $\text{NOT } \text{NOT } A = A$                                                                                                                       |
| De Morgan         | $\text{NOT}(A \text{ AND } B) = \text{NOT } A \text{ OR } \text{NOT } B$ und $\text{NOT}(A \text{ OR } B) = \text{NOT } A \text{ AND } \text{NOT } B$ |
| Regel 16          | $A \text{ OR } (\text{NOT } A \text{ AND } B) = A \text{ OR } B$                                                                                      |

## Disjunktive Normalform (DNF)

Aus einer Wahrheitstabelle liest man für jede Zeile mit Ergebnis 1 einen Produktterm (Minterm) ab und verknüpft alle mit ODER. Variablen mit Wert 0 werden negiert, mit Wert 1 nicht.

Beispiel (Alarm):  $\text{Alarm} = \text{NOT } A \text{ NOT } B \text{ C OR NOT } A \text{ B NOT } C \text{ OR NOT } A \text{ B C OR A NOT } B \text{ NOT } C \text{ OR ...}$  (vollständige disjunktive Normalform).

## Vereinfachung (algebraisch)

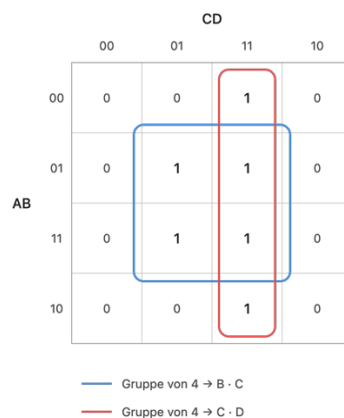
Mit den Rechengesetzen lassen sich Terme zusammenfassen. Grundidee: Es fällt weg, was sich ändert. Tritt eine Variable in zwei verknüpften Termen einmal negiert und einmal nicht auf, kann sie ausgeklammert werden.

## Karnaugh-Veitch-Diagramm (KV-Diagramm)

Das KV-Diagramm ist eine grafische Vereinfachungsmethode. Regeln zur Blockbildung:

- Die Anzahl der Felder in einem Block entspricht einer Zweierpotenz (1, 2, 4, 8 ...).
- Blöcke müssen Rechteckform haben.
- Möglichst wenige und möglichst grosse Blöcke wählen.
- Ein Block darf keinen anderen Block vollständig überdecken (sonst überflüssig).

Aus jedem Block liest man den gemeinsamen Term ab - die sich ändernden Variablen fallen weg.



## Gatterschaltungen & Schaltbilder

| Gatter      | Symbol-Kennung           | Funktion |
|-------------|--------------------------|----------|
| Negator     | 1 (mit Kreis am Ausgang) | NOT A    |
| UND-Gatter  | &                        | A AND B  |
| ODER-Gatter | >=1                      | A OR B   |

Schalter-Analogie: UND = Reihenschaltung, ODER = Parallelschaltung. Ein offener Schalter (Ruhekontakt) entspricht NOT A, ein geschlossener A.

## 8. Rechnerarchitektur (Von-Neumann)

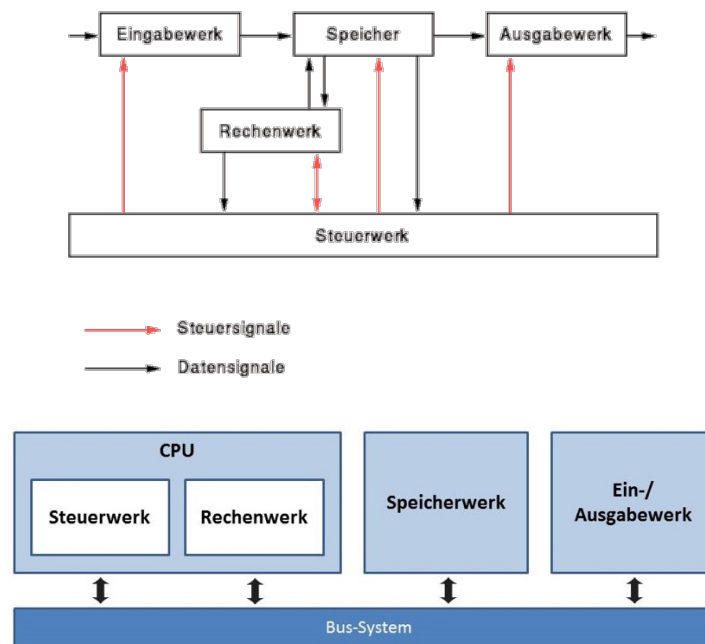


Abb.: Von-Neumann-Architektur

Die Von-Neumann-Architektur ist das Grundmodell des digitalen Rechners.

Charakteristisch: Programme und Daten liegen im gleichen Speicher.

| Einheit          | Aufgabe                                          |
|------------------|--------------------------------------------------|
| Eingabewerk      | Eingabe von Programm und Daten                   |
| Speicherwerk     | Haupt-/Arbeitsspeicher für Programme und Daten   |
| Rechenwerk (ALU) | führt arithmetische und logische Operationen aus |
| Steuerwerk       | steuert den Ablauf über Steuersignale            |
| Ausgabewerk      | Ausgabe der Ergebnisse                           |

Rechenwerk + Steuerwerk + Register bilden den Prozessor; Prozessor + Speicherwerk bilden die Zentraleinheit.

### Register & Sonderregister

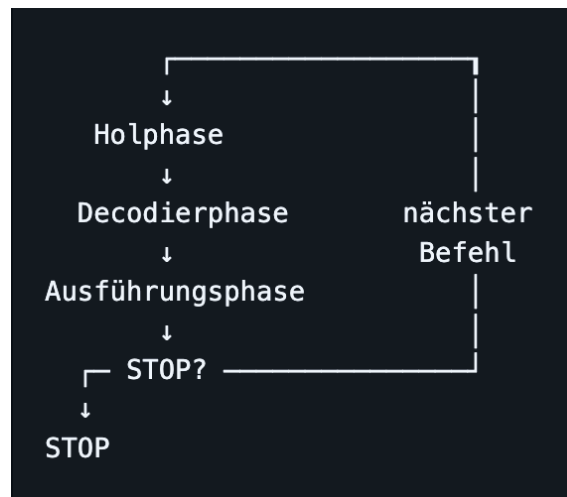
Register sind Speicherzellen direkt auf der CPU zur Aufnahme von Statusinformationen, Steuerinformationen (z. B. Befehlszähler) und Operanden.

**Sonderregister:** Befehlszähler und Akkumulator. Der Akkumulator nimmt Operanden, Zwischen- und Endergebnisse auf.

## Steuerwerk - der Befehlszyklus

Das Steuerwerk arbeitet jeden Befehl in drei Phasen ab (Schleife bis STOP):

- **Holphase:** Transport des nächsten Befehls aus dem Speicher ins Steuerwerk.
- **Decodierphase:** Entschlüsseln und Interpretieren des Befehls.
- **Ausführungsphase:** Erzeugung der Steuersignale zur Ausführung (z. B. durch Mikroprogramme).



## 9. CISC, RISC & Pipelining

|             | CISC                                                 | RISC                                                                                  |
|-------------|------------------------------------------------------|---------------------------------------------------------------------------------------|
| Name        | Complex Instruction Set Computer                     | Reduced Instruction Set Computer                                                      |
| Befehlssatz | umfangreich, komfortabel                             | reduziert, homogen                                                                    |
| Beispiel    | Intel-Prozessoren                                    | ARM-Prozessoren                                                                       |
| Eigenschaft | Befehle dauern unterschiedlich lange (mehrere Takte) | wesentlich weniger Taktfrequenz (ein Takt pro Befehl), weniger Wärme, weniger Kühlung |

### Phasen-Pipelining

Beim Pipelining werden Befehlsphasen überlappend ausgeführt, um die Programme zu parallelisieren. Während ein Befehl ausgeführt wird, wird der nächste bereits geholt usw.

- IF - Instruction Fetch (Befehl holen).
- DEC - Decode (decodieren).
- OF - Operand Fetch (Operanden holen).
- EXEC - Execute (ausführen).
- WB - Write Back (Ergebnis zurückschreiben).

Idealerweise wird pro Taktzyklus eine Phase abgearbeitet, sodass mehrere Befehle gleichzeitig in unterschiedlichen Phasen stecken.

## 10. Speicherarten

- RAM (Random Access Memory):  
Arbeits-/Hauptspeicher; wahlfreier Zugriff.      Flüchtig - Inhalt geht beim Abschalten verloren.

| Typ    | Eigenschaft                                                                                       |
|--------|---------------------------------------------------------------------------------------------------|
| ROM    | Read-Only-Memory; enthält unveränderbare Daten/Mikroprogramme für die Funktionalität des Systems. |
| PROM   | Programmierbarer ROM - einmal beschreibbar, nicht löschtbar.                                      |
| EPROM  | Eraseable PROM - mit UV-Licht löschtbar.                                                          |
| EEPROM | Electrically Eraseable PROM - elektrisch löschtbar; Basis von Flash-Speicher (SSD, USB-Sticks).   |

### Pufferspeicher (Caches)

- Werden gebraucht, wenn Daten zwischen unterschiedlich schnellen Betriebsmitteln ausgetauscht werden.
- Dienen dem Angleichen der Geschwindigkeiten (Transportgeschwindigkeit).
- Prinzip: langsam aufnehmen / schnell abgeben und umgekehrt.

### Virtueller Speicher

Der virtuelle Speicher entspricht dem gesamten Adressraum unabhängig vom physischen RAM. Er besteht aus realem Speicher (RAM) und Hintergrundspeicher/Sekundärspeicher (z. B. Festplatte).

Die Verwaltung erfolgt seitenweise: Eine Seitenadresse + Offset ergibt die physische Adresse. Über eine Seitentabelle wird geprüft, ob eine Seite im RAM liegt; falls nicht, muss sie nachgeladen werden.

### Wechselstrategien (Verdrängung)

| Strategie | Bedeutung                                                              |
|-----------|------------------------------------------------------------------------|
| FIFO      | First-In / First-Out - das älteste Element wird verdrängt.             |
| LIFO      | Last-In / First-Out - das zuletzt geladene Element wird verdrängt.     |
| LRU       | Least Recently Used - das am längsten nicht gebrauchte wird verdrängt. |
| LFU       | Least Frequently Used - das am seltensten genutzte wird verdrängt.     |

## 11. Bus & Geräteverwaltung

### Bus (Sammelleitung)

Ein Bus ist ein von mehreren Komponenten gemeinsam genutztes Übertragungsmedium. Dadurch besteht Kollisionsgefahr.

- Kollisionsbehandlung: CSMA/CD (erkennen und neu senden).
- Kollisionsvermeidung: Token / Controller (nur wer berechtigt ist, sendet).

| Übertragung | Eigenschaft                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------|
| Seriell     | Bits nacheinander -> erlaubt hohe Taktfrequenz.                                                               |
| Parallel    | mehrere Bits gleichzeitig über mehrere Leitungen -> Gefahr des Übersprechens, dadurch geringere Taktfrequenz. |

### Geräteverwaltung (E/A-Systeme)

Spezielle E/A-Prozessoren (Kanäle) sind zwischen CPU und Gerät geschaltet und übernehmen die Gerätebedienung selbstständig und parallel zur CPU. Die CPU gibt dem Kanal nur den Auftrag; der Kanal holt sich die Daten selbst.

**Cycle Stealing:** CPU und Kanal können nicht gleichzeitig auf denselben Speicher zugreifen. Bei einem Kanalzugriff wird der CPU ein Speicherzyklus gestohlen.

| Register            | Inhalt             |
|---------------------|--------------------|
| 1. E/A-Register     | Daten              |
| 2. Kontrollregister | Gerät, Fehler      |
| 3. Adressregister   | nächste Adresse    |
| 4. Längenzähler     | Rest = 0 -> fertig |

- Multiplexkanal: viele langsame Geräte gleichzeitig.
- Selektorkanal: ein schnelles Gerät.
- **Pufferung:** Daten sammeln und en bloc senden -> Effizienz.
- **Spooling:** Aufträge vormerken (z. B. Drucker), Ausgabe erfolgt, wenn das Gerät frei ist -> kein Warten.

## 12. Softwarekomponenten & Betriebssysteme

Aufgaben eines Betriebssystems: Ressourcenverwaltung und Aufrechterhaltung der Betriebsart.

| Kriterium | Varianten                                                                   |
|-----------|-----------------------------------------------------------------------------|
| Tasking   | Single-Tasking (eine Aufgabe) vs. Multitasking (mehrere Aufgaben parallel). |
| User      | Single-User vs. Multi-User.                                                 |

### Programmgruppen im BS:

**Organisationsprogramme** (Speicher-, Prozessor-, Geräteverwaltung, Kommunikation),

**Übersetzungsprogramme** (Interpreter, Compiler) und

**Dienstprogramme** (Lader, Linker/Binder, Editoren).

### Vom Quellcode zum Programm

Quellprogramm -> Compiler -> Objektcode -> Linker -> ausführbares Programm.

- Compiler: übersetzt den gesamten Code, überprüft die Syntax.
- Objektcode: verschiebbarer Code ohne endgültige Adressierung.
- Linker: bindet Bibliotheksfunktionen ein.

| Bindeart           | Beschreibung                                                                                                              |
|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| Statisches Binden  | Bibliotheksfunktionen werden in das ausführbare Programm kopiert.                                                         |
| Dynamisches Binden | Verweis auf eine gemeinsam nutzbare Bibliothek im Speicher; die dynamische Bibliothek (.dll) muss zur Laufzeit vorliegen. |

### Prozessorverwaltung (Scheduling)

- Nutzt einen Prozess-Scheduler; Ziele sind echtzeit- oder durchsatzorientiert.
- Feste Priorität (oft bei Echtzeit): unterscheidet zwischen hoher und geringer Priorität.
- Dynamische Priorität: Durchsatzpriorität, berücksichtigt das Verhalten der Prozesse - hohe Aktivität führt zu hoher Priorität.

**Interpreter:** arbeitet nach dem Prinzip Lesen -> Prüfen -> Ausführen (Zeile für Zeile, im Gegensatz zum Compiler).

## 13. Dateisysteme

Eine Datei (File) ist ein Strom von Bytes. Das Dateiverwaltungssystem verwaltet den gesamten Speicherplatz auf Massenspeichern. Aufgaben: Auffinden von Dateien, Zuweisung freien Speichers, Freigeben und Löschen von Dateien.

### Deskriptoren

| Dateideskriptor (i-Node, FAT-Eintrag)                             | Datenträgerdeskriptor (Superblock, FAT)        |
|-------------------------------------------------------------------|------------------------------------------------|
| Dateiname; Adresse Datenträger; Adresse Datei; Organisationsform; | Datenträgername;                               |
| Dateistatus (geöffnet/geschlossen, Lese-/Schreibzugriff);         | Dateideskriptoren aller gespeicherten Dateien; |
| Dateieigentümer; Zugriffsbeschränkungen.                          | Größe des Speichers;                           |
|                                                                   | Listen freier und belegter Speichereinheiten.  |

### Hierarchische Dateisysteme

- Verzeichnisse bestehen aus Name und Verweis auf Dateideskriptoren.
- „cd“ . = Verweis auf sich selbst;
- „cd“ .. = Verweis auf das übergeordnete Verzeichnis.
- Relativer Pfad: ausgehend vom aktuellen Verzeichnis (z. B. cd Müller).
- Absoluter Pfad: ausgehend vom Wurzelverzeichnis (z. B. cd /home/Müller).

### Dateiorganisation

| Organisation                | Eigenschaft                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------|
| Sequentiell (Heap)          | Fortlaufender Strom von Datensätzen in Reihenfolge der Eingabe; nur sequentiell lesbar.                                |
| Indexiert                   | Zugriff über einen Schlüssel; über eine Indexdatei oder eine Berechnungsvorschrift (Hash) wird die Adresse ermittelt.  |
| Indexsequentiell            | Nach Merkmal geordnete Elemente, in Einheiten mit Schlüsseln gruppiert; Datensätze innerhalb sequentiell durchlaufbar. |
| Multilisten                 | Listen + Ankertabelle; Datensätze gleichen Merkmals verkettet, Verweis über Zeiger.                                    |
| Mehrstufige Indexstrukturen | Baumartig bei sehr grossen Dateien; Wurzel = Masterindex.                                                              |



## Dateisysteme unter Unix

Innerer Aufbau (logische Blöcke): Block 0 Bootblock, Block 1 Superblock, Blöcke 2...n i-Node-Liste, Blöcke n+1...m Datenblöcke.

Die **i-Node** enthält: Dateityp, Zugriffsrechte, Anzahl Referenzen (Links), Eigentümer, Gruppengehörigkeit, Länge in Bytes, Datum/Zeiten sowie direkte Verweise und Verweise auf Indirektionsblöcke. Beim Mounten wird der Superblock in den Hauptspeicher geladen.

z.B: **stat ~/Documents/Uni/MPP/Betriebssysteme-Rechnernetze.docx**

```
16777231 116868724 -rw-r--r-- 1 kangon staff 0 163129 "Jun 13 22:25:14 2026" "Jun 12 15:46:22 2026" "Jun 13 15:41:12 2026" "Jun 11 11:55:58 2026" 4096 320 0x40
```

## Arten von Dateien

- **Reguläre Dateien:** enthalten Datenbestände (Anwender-/Systemdaten und -programme), unstrukturierte Folge von Bytes - Programme müssen selbst strukturieren.
- **Dateiinhaltsverzeichnisse:** Datensatz, der das Inhaltsverzeichnis einer Gruppe von Dateien darstellt; nur von privilegierten Programmen erstellbar.
- **Spezielle Dateien:** stellen Bezug zu angeschlossenen Geräten her; präsentiert durch Namen und zwei Zahlen: major device number (Gerätetreiber) und minor device number (Gerätnummer innerhalb einer Treiberklasse).

## 14. Programmiersprachen

Programmiersprachen sind die Schnittstelle zwischen Benutzer und Maschine. Grobe Einteilung: Maschinenorientiert (Maschinencode, Assembler) und Problemorientiert (höhere Sprachen).

### Problemorientierte (höhere) Sprachen

Sie sind weitgehend maschinenunabhängig und besitzen einen definierten Kern, der auf allen Maschinen gleich ist.

| Paradigma            | Idee                                                                                                                               | Vertreter |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Imperativ            | <b>Folge von Anweisungen;</b> beschreibt, WIE das Ergebnis erzeugt wird (Weg im Vordergrund).                                      | C, Pascal |
| Funktional           | Berechnet Funktionen, die Eingaben auf Ausgaben abbilden; Beschreibung über Ausdrücke.                                             | LISP      |
| Deskriptiv           | Das <b>Ergebnis steht im Vordergrund;</b> man beschreibt Eigenschaften, die das Ergebnis erfüllen soll ( <b>Abfragesprachen</b> ). | SQL       |
| Prädikativ (logisch) | Beweis in einem System von <b>Fakten und Regeln</b> (Wissensbasis); Anfragen liefern <b>boolesche Antworten</b> (Expertensysteme). | PROLOG    |
| Objektorientiert     | Daten und Operationen werden <b>zu Objekten zusammengefasst;</b> Kommunikation über Botschaften.                                   | C++, Java |

## Maschinenorientierte Sprachen

Sie orientieren sich an der **vorliegenden Hardware** und richten sich nach dem **Befehlssatz des Zentralprozessors** (Vertreter: Assemblersprachen).

Vorteil: schnellere Laufzeiten; Einsatz in Chipkarten und kleinen Mikrocontrollern.

**Befehl:** kleinste, nicht weiter zerlegbare Einheit; besteht aus Operationscode (spezifiziert den Befehl) und Adressteil (spezifiziert den Operanden).

## Adressierungsarten

| Art         | Beschreibung                                                                                                                  |
|-------------|-------------------------------------------------------------------------------------------------------------------------------|
| Unmittelbar | Operand steht direkt im Adressteil des Befehls -> kein zusätzlicher Speicherzugriff nötig.                                    |
| Direkt      | Der Adressteil enthält die Adresse, die genau den Aufenthaltsort des Operanden kennzeichnet.                                  |
| Indirekt    | Der Adressteil enthält die Adresse einer Speicherzelle, in der erst die eigentliche Operandenadresse steht.                   |
| Symbolisch  | Speicherzelle erhält einen frei wählbaren Namen; beim Linken wird er durch die absolute Adresse ersetzt.                      |
| Indiziert   | Operandenadresse = Adressteil + Wert im Indexregister (Summe).                                                                |
| Relativ     | Berechnung relativ zur gerade bearbeiteten Adresse (Befehlszähler/Program Counter); dient dem Auffinden des nächsten Befehls. |
| Virtuell    | Es wird ein Speicherbereich ausserhalb des physischen Hauptspeichers angesprochen.                                            |

## 15. Sprachen, Grammatiken & Beschreibungsformen

| Begriff                                                           | Bedeutung                                     |
|-------------------------------------------------------------------|-----------------------------------------------|
| Syntax                                                            | Lehre vom Satzbau (Struktur).                 |
| Semantik                                                          | Inhaltliche Bedeutung.                        |
| Pragmatik                                                         | Erwartete Aktion / Wirkung.                   |
| Natürliche Sprachen                                               | Künstliche Sprachen                           |
| umfangreich, ständige Veränderungen -> nicht endlich beschreibbar | einfach, klare Regeln -> endlich beschreibbar |

## Grammatik $G = (T, V, S, P)$

| Element | Bedeutung                                                                              |
|---------|----------------------------------------------------------------------------------------|
| T       | Terminalsymbole - unveränderliche, unteilbare Bestandteile (z. B. if, else).           |
| V       | Nichtterminalsymbole - Zeichen(folgen), die nach Regeln der Grammatik gebildet werden. |
| S       | Startsymbol - Nichtterminal, aus dem alle Worte der Sprache abgeleitet werden.         |
| P       | Menge der Grammatikregeln.                                                             |

Eine Grammatikregel ( $l, r$ ) besteht aus linker Seite  $l$  (zu erklärende Konstrukt) und rechter Seite  $r$  (Erklärung).

## Backus-Naur-Form (BNF)

Die BNF beschreibt Syntax formal. Schreibweise:  $\langle \text{Nichtterminal} \rangle ::= \text{Erklärung}$ ,  $|$  trennt Alternativen,  $[ ]$  kennzeichnet Optionales.

Beispiel:  $\langle \text{Zeichenkette} \rangle ::= \langle \text{Zeichen} \rangle | \langle \text{Zeichen} \rangle \langle \text{Zeichenkette} \rangle$ ;  $\langle \text{Zeichen} \rangle ::= \langle \text{Buchstabe} \rangle | \langle \text{Sonderzeichen} \rangle | \langle \text{Ziffer} \rangle$ ;

Das Epsilon steht für das leere Wort. Syntaxdiagramme sind die grafische Entsprechung der BNF.

## Reguläre Ausdrücke

Eine reguläre Menge  $R$  über einem Alphabet  $A$  wird nach diesen Regeln gebildet:

- Leere Menge ist regulär.
- $\{\epsilon\}$  (Menge mit nur dem leeren Wort) ist regulär.
- $\{a\}$  ist regulär für jedes  $a$  aus  $A$ .
- Sind  $p$  und  $q$  reguläre Ausdrücke, dann auch:  $(p)^*$  (Konkatenation beliebig oft),  $(p \text{ ODER } q)$  (Vereinigung) und  $(pq)$  (Konkatenation).

## Chomsky-Hierarchie

| Typ       | Sprache             | Einschränkung                                                                   |
|-----------|---------------------|---------------------------------------------------------------------------------|
| Chomsky 0 | rekursiv aufzählbar | keine Einschränkungen                                                           |
| Chomsky 1 | kontextsensitiv     | kontextabhängige Regeln                                                         |
| Chomsky 2 | kontextfrei         | linke Seite ein einzelnes Nichtterminal                                         |
| Chomsky 3 | regulär             | stärkste Einschränkung (von regulären Ausdrücken/endlichen Automaten erfassbar) |

## 16. Automaten

Automaten sind mathematische Modelle von Geräten, die Eingaben in Ergebnisse umformen.

- Erkennende Automaten (Akzeptoren): stellen fest, ob ein Wort zu einer Sprache gehört.  
Anwendung: Prüfen der syntaktischen Richtigkeit von Programmen.
- Übersetzende Automaten (Transduktoren): berechnen aus dem Eingabewort ein Ausgabewort.  
Anwendung: Compiler (Syntax prüfen, Korrektheit feststellen, in andere Sprache übersetzen).

## Deterministischer endlicher Automat (DEA)

Formal:  $A = (I, O, Q, \delta, q_0, F)$

| Symbol   | Bedeutung                    |
|----------|------------------------------|
| I        | Eingabealphabet              |
| O        | Ausgabealphabet              |
| Q        | Zustandsmenge                |
| $\delta$ | Zustandsübertragungsfunktion |
| $\gamma$ | Ausgabefunktion              |
| $q_0$    | Startzustand                 |
| F        | Finalzustände (Endzustände)  |

Die Übertragungsfunktion wird als Tripel notiert: (Ausgangszustand, Zeichen, Zielzustand). Beim Akzeptor muss das ganze Wort abgearbeitet werden und in einem Finalzustand enden. Ein Fallenzustand wird nie wieder verlassen -> Wort wird nicht akzeptiert.

## Determinismus - die vier Begriffe

| Begriff              | Bedeutung                                                               |
|----------------------|-------------------------------------------------------------------------|
| Deterministisch      | Aus einer Ausgangssituation folgt genau eine Folgesituation.            |
| Nichtdeterministisch | Aus einer Ausgangssituation folgen mehrere Folgesituationen oder keine. |
| Determiniert         | Gleiche Eingabewerte liefern gleiche Ergebnisse.                        |
| Nicht determiniert   | Gleiche Eingabe kann unterschiedliche Ergebnisse liefern.               |

## Nichtdeterministischer Automat (NEA)

- Muss korrekte Worte akzeptieren.
- Darf bei fehlerhaften Worten in undefinierten Zuständen steckenbleiben.

Ein NEA akzeptiert ein Wort, wenn es mindestens einen Weg vom Startzustand zu einem Finalzustand gibt, bei dem das Wort vollständig verarbeitet wird.

## Umwandlung NEA -> DEA

- Einführung von Zwischenzuständen zur Aufspaltung von Silben.
- Bilden neuer (zusammengesetzter) Zustände aus Mengen erreichbarer Zustände (Potenzmengenkonstruktion).

## Grammatik aus einem endlichen Automaten ableiten

Aus  $M = (I, Q, q_0, \delta, F)$  lässt sich eine Grammatik  $G = (T, V, S, P)$  gewinnen:

- $T$  entspricht  $I$  (Terminalsymbole = Eingabealphabet).
- $V$  entspricht  $Q$  ohne die Fallenzustände (Nichtterminale = Zustände).
- $S$  entspricht  $q_0$  (Startsymbol = Startzustand).
- $P$  entspricht  $\delta$  ohne die Wege in die Falle; zusätzlich  $q_t \rightarrow \epsilon$  für jeden Finalzustand.

## 17. Turingmaschine

Die Turingmaschine arbeitet mit einem Schreib-/Lesekopf auf einem (beidseitig unendlichen) Band. Der Kopf kann sich nach L (links), R (rechts) bewegen oder stehen bleiben (N).

Formal:  $T = (I, B, Q, q_0, \delta, F)$

| Symbol   | Bedeutung                                             |
|----------|-------------------------------------------------------|
| I        | Eingabealphabet (z. B. $\{a, b\}$ )                   |
| B        | Bandalphabet (z. B. $\{a, b, \#\}$ mit Leerzeichen #) |
| Q        | Zustandsmenge                                         |
| $q_0$    | Startzustand                                          |
| $\delta$ | Zustandsüberföhrungsfunktion                          |
| F        | Finalzustände                                         |

Ein Übergang hat die Form  $\delta(q, x) = (q', x', d)$ : Im Zustand  $q$  wird Zeichen  $x$  gelesen, durch  $x'$  ersetzt, in Zustand  $q'$  gewechselt und der Kopf um  $d$  (L/R/N) bewegt.

## Konfiguration

Eine Konfiguration  $\alpha q \beta$  beschreibt den Gesamtzustand:  $\alpha$  = bereits verarbeitete Eingabe / erzeugte Ausgabe,  $q$  = aktueller Zustand,  $\beta$  = noch zu verarbeitende Eingabe. Ein Berechnungsschritt:  $\alpha q \beta \rightarrow \alpha' q' \beta'$ ; mehrere Schritte:  $\rightarrow^*$ .

Anwendungsbeispiele: Inkrementmaschine (erhöht eine Binärzahl um 1) sowie nichtdeterministische Turingmaschinen.

## 18. Registermaschine

Die Registermaschine berechnet eine Funktion  $f: N^r \rightarrow N^s$  ( $r$  Eingabewerte,  $s$  Ergebniswerte) und ist eng an die Von-Neumann-Idee angelehnt.

| Von-Neumann-Werk | Registermaschine                       |
|------------------|----------------------------------------|
| Speicherwerk     | Register                               |
| Rechenwerk       | Operationen $f, t$                     |
| Steuerwerk       | delta (Programmabarbeitungskomponente) |
| Eingabewerk      | erste $r$ Register                     |
| Ausgabewerk      | erste $s$ Register                     |

### Operationen

Die Registermaschine kennt nur drei elementare Operationen:

- Inkrement (A): Erhöhen eines Registerinhalts um 1 (Addition).
- Dekrement (S): Vermindern um 1, aber nicht unter 0 (Subtraktion):  $x-1$  falls  $x > 0$ , sonst 0.
- Test (T): Prüfen, ob der Inhalt eines Registers 0 ist.

### Programmaufbau

Ein Programm besteht aus nummerierten Anweisungen in zwei Formen:

- $i : DO f ; GOTO j \rightarrow$  abgekürzt  $(i, f, j)$  - führe Operation  $f$  aus, springe zu Zeile  $j$ .
- $i : IF t THEN GOTO j ELSE GOTO k \rightarrow$  abgekürzt  $(i, t, j, k)$ .

Formale Definition:  $R_m = (F, T, P, \text{delta})$ . Die Maschine startet mit  $\text{delta} = (0, x_1, \dots, x_r, 0, \dots, 0)$  und stoppt, wenn die Markierung das Programmende erreicht.

### Beispiel: Addition (Tupelschreibweise)

Programm: 0: IF T2 THEN GOTO 3 ELSE GOTO 1 - 1: DO A1 ; GOTO 2 - 2: DO S2 ; GOTO 0.

Idee: Register 2 wird so lange dekrementiert und Register 1 inkrementiert, bis  $R_2 = 0$  ist  $\rightarrow R_1$  enthält die Summe. Ablauf z. B.  $(0,5,3) \rightarrow \dots \rightarrow (3,8,0)$ : Start  $5 + 3$ , Ergebnis 8.